

# Introduction to Programming

## Problem Sheet 2: Solutions

---

Recursion Practice Problem Set All functions must be implemented using recursion (no built-in iteration). Use only Integer (no Int), no lists, only tuples where necessary. No imports

1. Factorial of a non-negative integer.

Compute  $n! = 1 \times 2 \times \dots \times n$ .

```
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n-1)
```

2. Power: raise x to the n-th power ( $n \geq 0$ ).

Compute  $x^n$  using only multiplication and recursion.

```
power :: Integer -> Integer -> Integer
power x 0 = 1
power x n = x * power x (n-1)
```

3. Sum of digits of a non-negative integer.

e.g., `sumDigits 123 = 6`.

```
sumDigits :: Integer -> Integer
sumDigits 0 = 0
sumDigits n = let (m,d) = n `divMod` 10
               in d + sumDigits m
```

4. Product of digits (non-negative integer).

e.g., `productDigits 234 = 24`.

```
productDigits :: Integer -> Integer
productDigits 0 = 1
productDigits n = let (m,d) = n `divMod` 10
                  in d * productDigits m
```

5. Sum of integers from a to b (inclusive), assuming  $a \leq b$ .

```
sumRange :: Integer -> Integer -> Integer
sumRange a b
  | a == b = a
  | a < b = b + sumRange a (b-1)
```

6. Count the number of digits in a non-negative integer.

e.g., `countDigits 0 = 1`, `countDigits 123 = 3`.

```
countDigits :: Integer -> Integer
countDigits n | n < 10 = 1
countDigits n = let m = n `div` 10
                in 1 + countDigits m
```

7. Check if a non-negative integer is even.

Use only recursion (no ``mod``, ``even``, or division).

```
isEven :: Integer -> Bool
isEven 0 = True
isEven 1 = False
isEven n = isEven (n-2)
```

8. Compute the nth triangular number:  $T(n) = 1 + 2 + \dots + n$ .

```
triangular :: Integer -> Integer
triangular 0 = 0
triangular n = n + triangular (n-1)
```

9. Fibonacci number (naïve double recursion).

Define `fib(0) = 0`, `fib(1) = 1`, and for  $n > 1$  use the sum of the two previous.

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

10. Binomial coefficient  $C(n,k)$  using Pascal's rule.

$C(n,k)$  counts the number of ways to choose  $k$  items from  $n$ .

```
binom :: Integer -> Integer -> Integer
binom n 0 = 1
binom 0 r = 0
binom n r = binom (n-1) (r-1) + binom (n-1) r
```

11. Convert a number from one base to another.

Given  $n$  (written in base  $b_1$ ) and target base  $b_2$  ( $2 \leq b_1, b_2 \leq 10$ ), return the representation of the same value in base  $b_2$  as an Integer whose decimal digits are the digits of the new representation.  
e.g., `baseConvert 123 4 8 = 33` (because  $123_4 = 27_{10} = 33_8$ )

```
baseConvert :: Integer -> Integer -> Integer -> Integer
baseConvert 0 _ _ = 0
baseConvert n b 10 = let (m,d) = n `divMod` 10
  in ( baseConvert m b 10 ) * b + d
baseConvert n 10 b = let (m,d) = n `divMod` b
  in ( baseConvert m 10 b ) * 10 + d
baseConvert n b1 b2 = baseConvert (baseConvert n b1 10) 10 b2
```

13. Power tower of height  $n$  with base  $n$ .

Compute  $n^{(n^{(\dots^n)})}$  where the tower has  $n$  occurrences of  $n$ .  
This function grows extremely fast using only multiplication and recursion.

```
powTow :: Integer -> Integer
powTow n = baseCount n n where
  baseCount x 0 = 1
  baseCount x n = power x (baseCount x (n-1))
```

14. Sum of two integers using only increment and decrement operations.

(No use of +, only successor and predecessor.)

```
addRec :: Integer -> Integer -> Integer
addRec 0 m = m
addRec n m = succ $ addRec (pred n) m
```

15. Multiply two integers using only addition and recursion.

Assume non-negative inputs.

```
mulRec :: Integer -> Integer -> Integer
mulRec 0 m = 0
mulRec n m = addRec m $ mulRec (pred n) m
```

16. Apply a function n times to an initial value.

e.g., `iterate' 3 succ 0 = 3`.

```
iterate' :: Integer -> (Integer -> Integer) -> Integer -> Integer
iterate' 0 f = id
iterate' n f = f . iterate' (n-1) f
```

17. Repeatedly apply a function until a condition holds.

Given f and p, keep applying f while p is false,  
return the first value satisfying p. (Assume termination.)

```
until' :: (Integer -> Bool) -> (Integer -> Integer) -> Integer -> Integer
until' stop f n = if stop n then n else until' stop f (f n)
```